Software Requirements Specification (SRS)

Event Registration & Ticket Payment Platform — University of Ibadan Technology Conference

Web Development, Group 5 · Version 1.0 · 8 October 2026 · Prepared by: Technical Lead / Architecture

---

0. Executive Summary

- **What:** A web platform where attendees register, pick tickets, pay, and receive a unique QR-style digital ticket. Gate officers verify tickets at the entrance. Admins see sales and attendance live.
- **Why:** The current process (forms, cash, spreadsheets) is slow, error-prone, and cannot stop fake or reused tickets.
- **How:** HTML/CSS/JavaScript frontend talking to a PHP API through the Fetch API, backed by MySQL. The server is the single source of truth for prices, payments, and ticket validity.
- **Who:** 8 people in 5 workstreams, delivering in 14 days.
- **Core design rule:** Never trust the browser. Totals, payment status, and ticket validity are always decided by the backend.

---

1. Introduction

1.1 Purpose

This document defines what the system must do (requirements), how it will be built (implementation strategy and tools), and how it will be proven correct (testing). It is the shared reference for all 8 team members, the lecturer/assessor, and the conference organisers.

1.2 Scope

In scope

- Event information page, user registration, login/logout
- Ticket categories, pricing, quantity selection, order summary
- Payment (real integration or controlled simulation) and transaction records
- Unique ticket generation with QR-style code, viewable and printable

- Ticket verification (valid / used / invalid) and reuse prevention
- Admin dashboard: sales, tickets per category, attendee list and search, price and event editing

Out of scope (prototype)

- Refunds and chargebacks, ticket transfers, multi-event support
- Email/SMS delivery (optional stretch)
- Native mobile apps, offline scanning

1.3 Definitions

Term	Meaning
Order	A purchase of one or more tickets by one user
Ticket	One admission right, with a unique number and verification code
Verification code	Random secret string encoded in the QR image; the thing scanned at the gate
Ticket number	Human-readable ID printed on the ticket (for manual lookup)
API envelope	The consistent JSON wrapper every endpoint returns
Idempotent	Repeating the same request does not create duplicate results

1.4 Assumptions, Risks, and Dependencies (flagged up front)

ID	Assumption / Risk	Impact	Mitigation
A1	Payment will be a **controlled simulation** by default; a real gateway (e.g. a test-mode Nigerian provider) is a stretch goal	Real gateways need accounts, keys, webhooks	Build payment behind one module so the gateway can be swapped without touching other code
A2	One event only (the UI Tech Conference)	Simplifies schema	Keep an events table so multi-event is possible later

ID	Assumption / Risk	Impact	Mitigation
A3	Roles are attendee, admin, verifier, stored on the users table	Needed to protect admin and gate functions	Seed one admin and one verifier account
A4	QR codes are generated in the browser with a small JS library loaded from a CDN	Needs internet at demo time	Bundle the library locally as a fallback
A5	Team uses XAMPP/WAMP or Laragon locally, with one shared Git repository	Environment drift	Provide a single setup guide and SQL seed file (Section 9)
R1	Parallel work can break on mismatched API shapes	Integration delays	Freeze API contracts on Day 2 (Section 7)
R2	Two people could buy the last tickets at the same moment	Overselling	Row locking inside a transaction (Section 8.4)
R3	Two officers could scan the same ticket at the same moment	Double entry	Atomic conditional update (Section 8.5)
R4	Two-week deadline	Scope creep	Must-have list in Section 12; stretch goals only after Day 11

---

2. Overall Description

2.1 System Context

Browser (attendee, admin, officer) → JavaScript Fetch → PHP API → MySQL. A payment provider (or simulator) is called only by the backend.

2.2 User Classes

User	Goal	Key permissions
Attendee (student, staff, public, VIP)	Buy and hold a ticket	Register, log in, order, pay, view own tickets
Verification officer	Admit people at the gate	Verify tickets, mark as used

User	Goal	Key permissions
Administrator / organiser	Run the event	Everything an officer can do, plus dashboard, prices, event info

2.3 Operating Environment

Current Chrome, Firefox, Edge, and mobile browsers. PHP 8.x, MySQL 8.x (or MariaDB), Apache.

2.4 Constraints

- Backend must be PHP; frontend-API communication must use the Fetch API; storage must be MySQL.
- Complete within two weeks; modules must allow parallel development.
- Payment and QR must stay practical for a student prototype.

---

3. Team Structure and Responsibilities

The slide deck defines 5 workstreams and 8 people. The split of the 2-person streams below is a **suggested** division so nobody overlaps; adjust in the Day 1 meeting.

#	Workstream	People	Owns	Waits on
1	Project Management & Documentation	1 (PM)	SRS, diagrams (ERD, use case, flow), API contract doc, daily stand-up, risk log, final report	Nothing; unblocks everyone
2	UI/UX & Frontend	2 (FE-A, FE-B)	**FE-A:** event page, register/login, ticket selection, order summary, checkout. **FE-B:** digital ticket page (QR, print), admin UI, verification UI, shared fetch helper and styling	API contracts (Day 2), then live endpoints

#	Workstream	People	Owns	Waits on
3	Backend & Database	2 (BE-A, BE-B)	**BE-A:** schema, seed data, DB connection, registration, login, sessions, role checks. **BE-B:** shared helpers (JSON responses, validation, error handling), event and category endpoints, admin endpoints	Agreed ERD (Day 2)
4	Ticketing, Payment & Verification	2 (TP-A, TP-B)	**TP-A:** pricing rules, order creation, payment module, transactions. **TP-B:** ticket number and code generation, QR representation, verify-ticket logic, reuse prevention	Schema, helpers, category prices
5	Admin & Testing	1 (QA)	Test plan, test cases, bug tracker, dashboard queries validation, end-to-end tests	Completed modules

Rule from the project brief: nobody waits for an entire role to finish. They wait only for the specific interface or data they need. Use stub (fake) JSON responses until the real endpoint is ready.

---

4. System Architecture

4.1 Layers

Layer	Technology	Responsibility
Presentation	HTML, CSS, JavaScript	Pages, forms, live total estimate, QR drawing
Communication	Fetch API, JSON	Every call to the backend
Application	PHP API (modular files)	Validation, pricing, payment, ticketing, auth
Data	MySQL via PDO	Persistent, transactional storage

4.2 Why this design (talking points)

- **Thin client, authoritative server.** The browser shows an estimated total for speed, but the server recomputes it. This blocks price tampering through DevTools.
- **One envelope for all responses.** Frontend developers write one error handler instead of one per endpoint.
- **Payment before ticket, in one transaction.** A ticket row can never exist without a confirmed payment row.
- **Prepared statements everywhere.** Closes the most common student-project hole: SQL injection.
- **Modular PHP files.** Each endpoint is small and owned by one person, which reduces merge conflicts.

4.3 High-Level Flow

1. Visitor views event information.
2. Visitor registers or logs in.
3. User selects category and quantity; browser shows an estimate.
4. User confirms; server creates a **pending** order and recomputes the total.
5. User reviews the summary and pays.
6. Server confirms payment, marks the order **paid**, and creates tickets (single transaction).
7. Each ticket gets a ticket number and verification code; the QR encodes the code.
8. At the gate, an officer submits the code.
9. Valid ticket becomes **used**; used or unknown tickets are rejected.

4-step rule (NFR): Step 1 Event page (with login prompt) → Step 2 Select tickets → Step 3 Review and pay → Step 4 Confirmation with ticket.

---

5. Functional Requirements

Priority: **M** = must have, **S** = should have. Each requirement has an acceptance criterion testers can check.

5.1 User and Purchase Flow

ID	Requirement	Priority	Acceptance criterion
FR01	Display event title, date, venue, schedule, speakers	M	Page loads data from the event endpoint and shows all five items

ID	Requirement	Priority	Acceptance criterion
FR02	Register with name, email, phone, password, attendee type	M	Valid data creates an account; duplicate email is rejected with a clear message
FR03	Log in and log out	M	Correct credentials start a session; logout ends it; protected pages redirect when logged out
FR04	Show ticket categories and prices	M	All active categories appear with current prices from the database
FR05	Select category and quantity	M	Quantity must be a whole number between 1 and the per-order maximum
FR06	Compute total in the browser; re-check on the server	M	Browser total equals server total in normal use; tampered totals are ignored and the server value is used
FR07	Show order summary before payment	M	Summary lists category, quantity, unit price, line totals, grand total
FR08	Process payment and record the transaction	M	A payment row with a unique reference is stored for every attempt

5.2 Ticket and Admin Flow

ID	Requirement	Priority	Acceptance criterion
FR09	Show payment confirmation (success or failure)	M	Clear message and next action for each outcome
FR10	Generate a unique ticket number per ticket	M	No two tickets share a number (database unique constraint)
FR11	Display a QR-style digital ticket, viewable and printable	M	Ticket page shows holder, category, number, QR; print view is clean
FR12	Verify by number or code; return valid, used, or invalid	M	Each of the three outcomes is returned correctly, with holder and category when found

ID	Requirement	Priority	Acceptance criterion
FR13	Mark ticket used after successful entry verification	M	Second verification of the same ticket returns used
FR14	Admin dashboard: sales totals, tickets per category, attendee list, search, price edit, event edit	M	Figures match database queries; edits persist; non-admins are blocked

5.3 Proposed Additions (needed to make the brief work)

ID	Requirement	Priority	Reason
FR15	Admin and verifier login with role check	M	The brief requires admin authentication; roles make it enforceable
FR16	My Tickets page for logged-in attendees	S	Users need to reopen tickets after purchase
FR17	Ticket quota per category with sold-out message	S	Prevents overselling
FR18	Verification log (who scanned what, when, result)	S	Audit trail and demo evidence

---

6. Non-Functional Requirements

ID	Area	Requirement	How we meet and check it
NFR01	Security	Passwords hashed	PHP password_hash and password_verify; QA checks no plain text in database
NFR02	Security	SQL injection resistance	PDO prepared statements only; code review checklist
NFR03	Security	Server-side validation	Every endpoint validates type, length, range; QA sends bad input
NFR04	Security	Server-side pricing	Price read from database, never from request

ID	Area	Requirement	How we meet and check it
NFR05	Security	Admin protection	Session plus role check at the top of every admin endpoint
NFR06	Integrity	No ticket without confirmed payment	Single database transaction (Section 8.3)
NFR07	Performance	Page load under 3 s; API under 2 s (normal conditions)	Browser DevTools network tab; Lighthouse
NFR08	Usability	Mobile responsive; clear error messages	Test at phone width; every error has human text
NFR09	Usability	Max 4 steps to purchase	Walkthrough test
NFR10	Maintainability	Modular PHP; consistent JSON	Shared helper files; envelope check in tests
NFR11	Compatibility	Chrome, Firefox, Edge, mobile browsers	Browser matrix test

Extra security notes worth stating to stakeholders

- Use session cookies with HttpOnly and SameSite settings; regenerate the session ID on login.
- Escape all user text before displaying it (prevents script injection).
- Verification codes must be random (random_bytes), not sequential or guessable.
- Do not store card numbers. In simulation mode there are none; with a real gateway, only store the provider reference.

---

7. Data and API Design

7.1 Database Tables (freeze on Day 2)

Table	Key columns
events	id, title, event_date, venue, description, schedule, speakers
users	id, full_name, email (unique), phone, password_hash, attendee_type (student, staff, public, vip), role (attendee, admin, verifier), created_at

Table	Key columns
ticket_categories	id, event_id, name, price (DECIMAL 10,2), quota, is_active
orders	id, user_id, total_amount, status (pending, paid, failed, cancelled), created_at
order_items	id, order_id, category_id, quantity, unit_price (snapshot), line_total
payments	id, order_id, reference (unique), amount, method, status (success, failed), provider_response, created_at
tickets	id, order_id, category_id, ticket_number (unique), verification_code (unique), status (issued, used), issued_at, used_at, verified_by
verification_log	id, ticket_id (nullable), input_code, result, officer_id, created_at

Why unit_price is copied into order_items: if an admin changes prices later, old orders must still show what the customer actually paid.

Relationships: one user has many orders; one order has many items, many payment attempts, and many tickets; one category has many tickets.

7.2 Standard JSON Envelope (all endpoints)

Every response contains four fields: **success** (true or false), **message** (human-readable text), **data** (object or list, empty on failure), and **errors** (list of field-level problems, empty on success). HTTP status codes: 200 OK, 201 created, 400 validation failure, 401 not logged in, 403 wrong role, 404 not found, 409 conflict (duplicate or sold out), 500 server error.

7.3 Endpoint Contract

Endpoint	Method	Auth	Input	Output (data)	Owner
/api/event.php	GET	None	none	event details	BE-B
/api/categories.php	GET	None	none	categories with prices and availability	BE-B

Endpoint	Method	Auth	Input	Output (data)	Owner
/api/register.php	POST	None	name, email, phone, password, attendee_type	new user id	BE-A
/api/login.php	POST	None	email, password	user name and role	BE-A
/api/logout.php	POST	User	none	confirmation	BE-A
/api/order.php	POST	User	items (category_id, quantity)	order id, server-computed total, lines	TP-A
/api/payment.php	POST	User	order_id, payment details or simulated outcome	payment status, receipt, reference	TP-A
/api/ticket.php	GET	User (owner)	order_id	tickets with number, code, category	TP-B
/api/verify-ticket.php	POST	Verifier or admin	ticket number or code	result (valid, used, invalid), holder, category	TP-B
/api/admin/stats.php	GET	Admin	optional filters	sales total, tickets per category, counts	BE-B
/api/admin/attendees.php	GET	Admin	search text, page	attendee list	BE-B
/api/admin/update-price.php	POST	Admin	category_id, new price	updated category	BE-B
/api/admin/update-event.php	POST	Admin	event fields	updated event	BE-B

The four core endpoints from the brief (register, payment, ticket, verify-ticket) are kept as named; the others are the additional endpoints the brief allows.

---

8. Implementation Strategy

8.1 Tools

Purpose	Tool
Local server	XAMPP, WAMP, or Laragon (PHP 8.x, MySQL, Apache)
Editor	VS Code with PHP and live-server extensions
Version control	Git and GitHub (one repo, feature branches, pull requests)

Purpose	Tool
Database design	MySQL Workbench or dbdiagram.io for the ERD; phpMyAdmin for inspection
API testing	Postman or Thunder Client
QR drawing	Small client-side JS QR library (bundle locally)
Bug tracking	GitHub Issues with labels (bug, severity, module)
Diagrams	draw.io or Mermaid
Communication	One group chat plus a 10-minute daily stand-up

8.2 Suggested Project Structure

- **/api/** — endpoint files; **/api/admin/** — admin endpoints
- **/includes/** — db.php (PDO connection), response.php (envelope helper), auth.php (session and role checks), validate.php (input helpers)
- **/public/** — HTML pages, **/public/css/**, **/public/js/** (api.js with the shared fetch wrapper, plus one JS file per page)
- **/database/** — schema.sql and seed.sql (categories, event, admin and verifier accounts)
- **/docs/** — SRS, ERD, API contract, test plan
- **/tests/** — Postman collection and test log

Git rules: never commit directly to main; one branch per feature; small pull requests reviewed by one other person; config with database credentials stays out of Git (commit an example file only).

8.3 Order and Payment Transaction (critical design)

Steps inside one database transaction after payment is confirmed:

1. Lock the order and check it is still pending (prevents paying twice).
2. Lock the category rows and check remaining quota.
3. Insert the payment row with its unique reference.
4. Set the order to paid.
5. Insert one ticket row per quantity, each with a generated number and code.
6. Commit. If anything fails, roll back everything and return a failure message.

Why: the brief says a ticket must not exist without confirmed payment. A transaction makes this all-or-nothing, so a crash halfway cannot leave a half-finished order.

Idempotency: if the same payment reference arrives twice, return the existing result instead of creating new tickets.

8.4 Pricing and Quota

- Browser: multiply price by quantity for instant feedback only.
- Server: read prices from the database, multiply, sum, store. Reject quantity below 1 or above the per-order cap (suggest 10).
- Quota check uses row locking (SELECT … FOR UPDATE) inside the transaction so the last ticket cannot be sold twice.

8.5 Ticket Generation and Verification

- **Ticket number:** readable format such as UIT26 followed by a short random block (for manual lookup).
- **Verification code:** long random string from random_bytes, stored with a unique constraint. This is what the QR encodes.
- **QR representation:** the browser draws the code as a QR image. Printing uses a print stylesheet.
- **Marking as used (reuse prevention):** run one update that sets the ticket to used only where the code matches and status is still issued. If exactly one row changed, the answer is valid; if none changed, look up the ticket to answer used or invalid. Because the check and the change are one statement, two simultaneous scans cannot both succeed.

8.6 Payment Module Strategy

- **Default (prototype):** simulated gateway. The payment page offers a test card form or Succeed/Fail buttons; the server records the outcome like a real provider would.
- **Stretch:** real gateway in test mode. Because the module is isolated, only the inside of payment.php changes.
- Always record failed attempts; they are useful for admin reports and demos.

8.7 Frontend Strategy

- One shared api.js wrapper: sets headers, parses JSON, handles the envelope, shows errors consistently.
- Pages: index (event), register, login, tickets (select), checkout, ticket view, my tickets, admin dashboard, verify.
- Keep the browser total clearly labelled estimated until the server confirms.
- Mobile first CSS; large tap targets on the verify page (officers will use phones).

8.8 Coding Standards

- Every endpoint: check method, check auth/role, validate input, run prepared query, return envelope.
- Meaningful names; one responsibility per file; comments only where the why is not obvious.
- Never echo raw database errors to the user; log them, show a friendly message.

8.9 Per-Workstream Task Checklist

Stream	Key tasks
PM	Finalise SRS; ERD and API contract by Day 2; daily stand-up; track blockers; compile final report and demo script
FE (2)	Design system and wireframes; static pages with stub data; connect to endpoints as they land; QR and print view; admin and verify screens
BE (2)	Schema and seed; connection and helpers; auth and roles; event, category, admin endpoints; validation
Ticketing (2)	Pricing and order logic; payment module; transaction; ticket generation; verification with reuse prevention
QA (1)	Test plan on Day 2; test data; test each module on arrival; log bugs; full regression before submission

---

9. Environment Setup Guide (for all members)

1. Install XAMPP or Laragon; start Apache and MySQL.
2. Clone the repository into the web root folder.
3. Create the database in phpMyAdmin and import schema.sql then seed.sql.
4. Copy the example config file, fill in local database credentials.
5. Open the site in the browser; confirm the event endpoint returns the standard envelope.
6. Import the shared Postman collection.

Seed accounts: one admin, one verifier, one attendee, and three ticket categories (for example Student, General, VIP) so everyone tests against the same data.

10. Testing Strategy

10.1 Levels

Level	What	Who	Tool
Unit / module	Pricing calculation, code generation, validators	Module owner	PHP test scripts or PHPUnit (optional)
API	Each endpoint, valid and invalid input, auth	QA with BE and TP	Postman
Integration	Frontend to API to database	FE with BE	Browser plus DevTools
End-to-end	Register to pay to ticket to verify	QA	Manual script
Security	Injection, tampering, role bypass	QA	Manual attempts, Postman
Usability and compatibility	Mobile, browsers, 4-step rule	QA and FE	Real devices, Lighthouse

Rule: the developer tests their own work first; QA tests after a module is marked ready, not before.

10.2 Core Test Cases

ID	Covers	Scenario	Expected result
T01	FR02	Register with valid data	Account created, success message
T02	FR02	Register with existing email	409, clear message, no duplicate row
T03	FR02, NFR03	Register with bad email, short password, empty name	400 with field-level errors
T04	NFR01	Inspect users table after registration	Only hashed passwords present
T05	FR03	Login with wrong password	401, no session

ID	Covers	Scenario	Expected result
T06	FR03, NFR05	Open admin endpoint while logged out and as attendee	401 then 403
T07	FR05	Quantity 0, negative, decimal, text, very large	Rejected with message
T08	FR06, NFR04	Send tampered total or price in the request	Server ignores it and returns the database-based total
T09	FR06	Multiple categories in one order	Server total equals sum of line totals
T10	FR08, FR09	Successful simulated payment	Payment row, order paid, confirmation shown
T11	FR09	Failed payment	Order not paid, no tickets, clear failure message
T12	NFR06	Force an error midway through ticket creation	Full rollback; no payment-less tickets
T13	FR08	Repeat the same payment reference	No duplicate tickets (idempotent)
T14	FR10	Order of 5 tickets	Five distinct ticket numbers and codes
T15	FR11	Open ticket page; print preview	QR visible; print layout clean
T16	FR12	Verify a fresh ticket	valid with holder and category
T17	FR13	Verify same ticket again	used
T18	FR12	Verify random or edited code	invalid
T19	R3	Two simultaneous verifications of one ticket	Exactly one valid, one used
T20	R2, FR17	Two buyers race for the last ticket	One succeeds, one gets sold-out
T21	FR14	Compare dashboard totals with direct database queries	Figures match
T22	FR14	Admin changes a price	New price shown to buyers; old orders keep old price
T23	NFR02	Enter SQL-style text in text fields and search box	Treated as plain text; no error leak

ID	Covers	Scenario	Expected result
T24	Security	Enter script tags as name	Displayed as plain text, not executed
T25	NFR08, NFR11	Run full purchase on phone width and on three browsers	Works and is readable
T26	NFR07	Measure page and API times	Within 3 s and 2 s targets
T27	NFR09	Count steps from event page to ticket	4 or fewer

10.3 Edge Cases to Keep in Mind

- User closes the tab after paying; tickets must still appear in My Tickets.
- Payment succeeds but the response is lost; retry must not double-charge or double-issue.
- Admin lowers a price while a user is in checkout; the server total at order creation wins and the summary shows it.
- A category is deactivated mid-checkout; order creation must reject it politely.
- Expired session during payment; redirect to login and keep the order.
- Ticket scanned by a user with only a ticket number typed in manually.

10.4 Bug Handling

Severity	Meaning	Target
Critical	Breaks purchase, ticket issuing, or verification; security hole	Fix same day
Major	Feature wrong but a workaround exists	Fix within 24 hours
Minor	Cosmetic or wording	Fix before submission if time allows

Every bug has: ID, module, steps to reproduce, expected versus actual, severity, owner, status. QA retests every fix and runs the regression list (T01 to T27) before the Day 12 freeze.

10.5 Exit Criteria (ready to submit)

- All must-have requirements pass their acceptance criteria.
- No open critical or major bugs.

- Full end-to-end run succeeds twice in a row on a clean database.
- Documentation and demo script reviewed.

---

11. Schedule and Dependencies

Days	Milestone	Key dependency
1	Project explained; workstreams assigned; SRS agreed	None
2	ERD, API contracts, UI structure frozen; test plan drafted	SRS
3–4	Database built; frontend foundation; authentication	ERD
4–6	Ticket selection, pricing, orders, payment	Schema, helpers, categories
6–7	QR and verification started; admin dashboard started	Paid orders exist
8–9	Payment, ticketing, admin modules complete; integration begins	All endpoints available
10–11	System testing and bug fixing	Integrated build
12	Final integration; documentation review; feature freeze	Bugs under control
13	Demo and presentation preparation	Stable build
14	Final review and submission	Exit criteria met

Critical path: ERD → backend helpers and auth → orders and payment → ticket generation → verification → end-to-end testing. Anything on this path gets priority and daily check-ins.

---

12. Prototype Acceptance (Definition of Done)

Brief criterion	Requirements	Tests
Register and log in	FR02, FR03	T01 to T06
Select ticket and quantity; correct total	FR04 to FR07	T07 to T09

Brief criterion	Requirements	Tests
Complete payment and receive confirmation	FR08, FR09	T10 to T13
Paid order produces unique digital ticket	FR10	T14
Ticket has QR-style code	FR11	T15
Admin verifies ticket and reuse is blocked	FR12, FR13	T16 to T19
Admin can view users, tickets, orders, payments	FR14	T21, T22
Full workflow works end to end	All	T25 to T27 plus end-to-end script

Stretch goals (only after Day 11): real gateway in test mode, email delivery of tickets, CSV export of attendees, verification log screen.

---

13. Presentation Cheat Sheet

For non-technical stakeholders

- Problem: slow, error-prone, fakeable tickets. Solution: buy online, get a unique QR ticket, scan at the gate.
- Every ticket can be used once; the system knows instantly if it is fake or already used.
- Organisers see live sales and attendance instead of waiting for spreadsheets.

For engineers and assessors

- Server is the source of truth: prices, payment state, ticket validity.
- Order, payment, and tickets are created in one transaction.
- Reuse prevention is one atomic update, so simultaneous scans cannot both pass.
- Prepared statements, hashed passwords, role checks, random verification codes.
- Modular endpoints and a single JSON envelope enable parallel work.

Likely questions

- Why simulate payment? Real gateways need accounts and webhooks; the module is isolated so a gateway can be swapped in.
- Can someone copy a QR image? Yes, but only the first scan is valid; later scans are rejected as used.
- What if the internet drops at the gate? Out of scope for the prototype; future work is offline verification with a synced list.

14. Document Control

Version	Date	Change
1.0	8 Oct 2026	First full SRS with implementation and testing guidance

Next steps: (1) Day 1 review: confirm the suggested split of the 2-person workstreams and the payment mode (simulate or real). (2) Freeze ERD and API contract. (3) QA turns Section 10 into a Postman collection and test log.